

MEMORY MANAGEMENT

Allocating, Managing, and Releasing System Resources in C++

System Programming Guide

Performance & Security Architecture

1. The Power of Direct Control

In many modern languages (like Java or Python), a "Garbage Collector" automatically manages memory. C++ is different. It gives you, the programmer, **direct manual control** over the computer's memory.

This "manual" nature is what makes C++ incredibly fast and efficient for building operating systems, high-end games, and browsers. However, with great power comes the responsibility of cleaning up after yourself.

Why Memory Management Matters:

- **Efficiency:** Use only the exact amount of RAM you need.
- **Performance:** Control when memory is allocated to avoid stuttering in real-time apps.
- **Stability:** Improper management leads to crashes and performance degradation via leaks.

There are two primary ways C++ handles memory: **The Stack** and **The Heap**.

2. Stack Memory

The Stack is a special area of memory used for **static memory allocation**. It follows the Last-In-First-Out (LIFO) principle, similar to a stack of plates.

```
void myFunction() {  
    int x = 10; // Allocated on the Stack  
} // x is automatically destroyed here
```

Characteristics of the Stack:

- **Automatic:** Variables are created when a function starts and deleted when it ends.
- **Speed:** Extremely fast because it behaves like a temporary scratchpad.
- **Fixed Size:** You must know the size of data at compile time.
- **Local Scope:** The variables only exist within curly braces {}.

When to use: Use the stack for temporary, small variables that don't need to exist after a function finishes.

3. Heap Memory

The Heap is used for **dynamic memory allocation**. Unlike the stack, the heap has no structured order and is significantly larger. However, memory here is not cleaned up automatically; you must "borrow" it and "return" it yourself.

Characteristics of the Heap:

- **Flexibility:** You can allocate memory of any size at any time (Runtime).
- **Persistence:** Memory stays allocated until you manually tell C++ to free it.
- **Access:** Must be accessed using **Pointers**.
- **Slower:** Finding a free spot on the heap takes slightly more time than stack operations.

Analogy: The Stack is like a waiter's tray (organized and temporary). The Heap is like a hotel storage room (unorganized but huge and permanent).

4. Allocation: The 'new' Keyword

To request memory from the heap, we use the new keyword. This tells the system: "Find me some space for this data type and give me its address."

```
// Allocate a single integer on the heap
int *ptr = new int;
```

```
// Assign a value to it
*ptr = 50;
```

```
cout << *ptr; // Outputs 50
```

Dynamic Arrays

This is where new shines. You can ask for an array of a size that the user chooses during the program execution.

```
int size;
cout << "Enter array size: ";
cin >> size;

int *arr = new int[size]; // Dynamic Array
```

5. Deallocation: The 'delete' Keyword

As soon as you are done with memory on the heap, you **must** release it back to the operating system using delete.

Freeing a Single Variable:

```
int *ptr = new int;  
// ... use ptr ...  
delete ptr; // Memory is now free
```

Freeing an Array:

If you used new[] (brackets), you must use delete[] to free it correctly.

```
int *arr = new int[10];  
// ... use arr ...  
delete[] arr; // Array is destroyed
```

Warning: Using delete on an array instead of delete[] is a common mistake that causes partial memory leaks.

6. Hazard: Memory Leaks

A **Memory Leak** occurs when you allocate heap memory using `new` but lose the pointer to it before calling `delete`. Since the pointer is gone, the memory is "lost" but technically still "in use."

```
void leakPower() {  
    int *p = new int(100);  
} // p dies, but the memory for 100 stays occupied forever!
```

Consequences:

- **Slowing Down:** As more leaks occur, available RAM decreases.
- **Crashes:** Eventually, the program or the entire OS will run out of memory and crash.

Best Practice: Every `new` must have a corresponding `delete`.

7. Hazard: Dangling Pointers

A **Dangling Pointer** is a pointer that still holds the address of memory that has already been deleted. Trying to access this address is dangerous.

```
int *p = new int(5);  
delete p; // Memory is freed  
  
cout << *p; // ❌ CRASH or GARBAGE DATA
```

The Solution: Resetting to nullptr

After deleting a pointer, always set it to `nullptr`. In C++, deleting a null pointer is safe and does nothing.

```
delete p;  
p = nullptr; // SAFE
```


8. Comparison: Stack vs. Heap

Feature	Stack Memory	Heap Memory
Management	Automatic (OS)	Manual (Programmer)
Speed	Very Fast	Slower
Accessibility	Local scope only	Global scope (via pointers)
Size	Limited (Smaller)	Huge (Limited by RAM)
Risk	Stack Overflow (Infinite loops)	Memory Leaks / Crashes

In modern C++, we often use **Smart Pointers** (like `std::unique_ptr`) to handle these `delete` calls automatically, but understanding the manual way is fundamental.

9. Summary & Best Practices

Mastering memory management separates professional C++ developers from beginners. Follow these rules to write robust code:

1. **Stack First:** If you don't need dynamic sizing, always prefer the Stack. It's faster and safer.
2. **RAII Pattern:** "Resource Acquisition Is Initialization." Ensure every resource you open/allocate is closed/freed.
3. **Mirror Rule:** For every new, write the delete immediately before you forget.
4. **Cleanup:** Always set pointers to `nullptr` after deleting them.
5. **Bounds Check:** Be careful not to go past the end of a dynamically allocated array.

Knowledge Check:

1. Which area of memory is faster, Stack or Heap?
2. What keyword is used to allocate an array of 50 floats at runtime?
3. Why should you use `delete[]` instead of `delete` for arrays?